FUNDAÇÃO ESCOLA DE COMÉRCIO ÁLVARES PENTEADO – FECAP
CIÊNCIA DA COMPUTAÇÃO – 6º SEMESTRE
SISTEMAS EMBARCADOS E ROBÓTICA

FLAVIA DA COSTA R. FARIA
GUILHERME MUNIZ GOMES
LUCAS MOREIRA DE GODOY
MARIA EDUARDA CABRAL FOLONI

SCORESMASH: MOLDURA DE DETECÇÃO DE IMPACTO PARA TREINOS DE TÊNIS

Entrega 2 – Programação e Controle de Dispositivos no Arduino
Projeto Interdisciplinar Smart Tennis Arena

SÃO PAULO
2026

1 INTRODUÇÃO

O ScoreSmash é uma moldura que identifica o ponto em que a bola de tênis atravessa um alvo e acende, em tempo real, o LED correspondente em uma matriz de 9 × 9. O sistema lê 16 sensores de luz (LDR) e controla 81 LEDs a partir de um Arduino Uno. Este relatório atende à Entrega 2 da disciplina, que pede um programa que leia ao menos um sensor e controle ao menos um atuador. O código-fonte completo e comentado está no Apêndice A.

2 SENSORES E COMPONENTES

Tabela 1 – Componentes utilizados

Componente	**Qtd.**	**Função**
Arduino Uno	1	Lê os sensores, executa a lógica e controla os LEDs
Laser 5 V	16	Emissor: 8 no topo (eixo X) e 8 à esquerda (eixo Y)
LDR + resistor 10 kΩ	16	**Sensor**: recebe o feixe do laser oposto (divisor de tensão)
Multiplexador 74HC4067	1	Liga os 16 LDRs a uma única entrada analógica (A0)
Registrador 74HC595	10	2 grupos de 5 CIs em cascata; cada CI controla uma fileira de LEDs
LED + resistor 330 Ω	81	**Atuador**: matriz 9 × 9 na parte de trás da moldura
Fonte 5 V / 2 A	1	Alimenta lasers, LDRs, CIs e LEDs, com GND comum ao Arduino

Fonte: elaborado pelos autores (2026).

LDR (sensor). É um resistor cuja resistência cai quando recebe luz. Como o Arduino mede tensão, cada LDR forma um divisor com um resistor de 10 kΩ, e a tensão lida em A0 é:

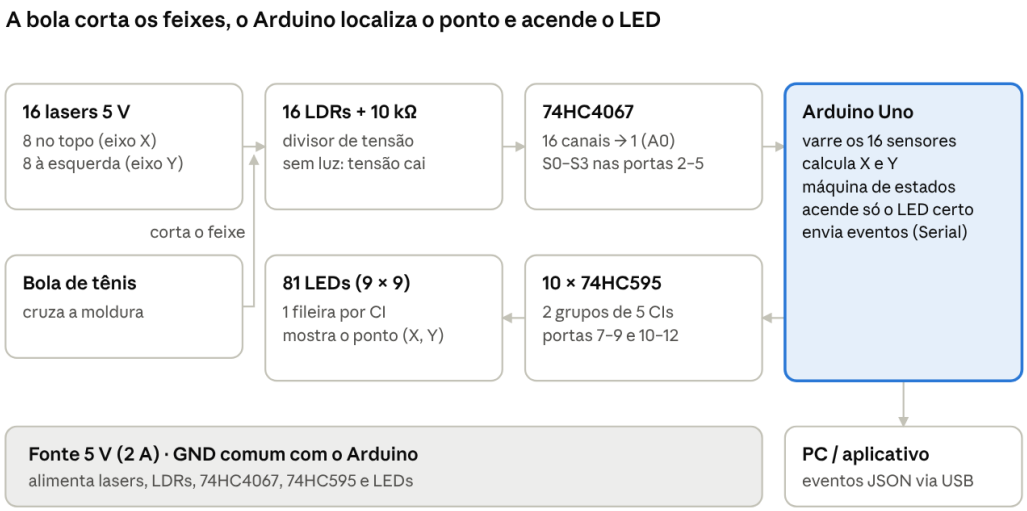
$$V_{out} = 5\ V \times 10\ k\Omega / (R_{LDR} + 10\ k\Omega)$$

Com o laser incidindo, R_{LDR} é baixa e a leitura fica alta (perto de 1023). Quando a bola corta o feixe, a resistência sobe e a leitura cai. Na inicialização, o firmware mede a leitura de cada sensor e considera o feixe cortado quando ela cai abaixo de 60% dessa referência, o que compensa a luz ambiente e o alinhamento de cada laser.

74HC4067 e 74HC595. O 74HC4067 seleciona, pelos pinos S0 a S3, qual dos 16 LDRs é ligado à entrada A0. O 74HC595 recebe 8 bits em série (dados + clock) e os apresenta em 8 saídas quando recebe o pulso de latch; ligados em cascata, cada grupo de 5 CIs forma um registrador de 40 bits, um bit por LED.

3 ARQUITETURA E LIGAÇÃO

Figura 1 – Diagrama de blocos do sistema



Fonte: elaborado pelos autores (2026).

Tabela 2 – Pinagem do Arduino Uno

Porta	Ligada em	Função
2, 3, 4, 5	S0, S1, S2, S3 do 74HC4067	Seleção do sensor (canal 0 a 15)
A0	SIG do 74HC4067	Leitura analógica do sensor
7 / 8 / 9	Pinos 11 / 12 / 14 do 1º CI do grupo A	Clock / latch / dados – fileiras 1 a 5
10 / 11 / 12	Pinos 11 / 12 / 14 do 1º CI do grupo B	Clock / latch / dados – fileiras 6 a 9 e 9ª coluna
GND	GND da fonte	Referência comum

Fonte: elaborado pelos autores (2026).

Em todos os 74HC595, o pino 13 (OE) fica no GND, o pino 10 (MR) no 5 V e o pino 9 (Q7') de cada CI vai ao pino 14 (DS) do CI seguinte. Os 10 CIs somam 80 saídas para 81 LEDs; por isso, o LED da fileira 9, coluna 9 fica sem saída.

APÊNDICE A – CÓDIGO-FONTE COMPLETO

Arquivo `scoresmash.ino` (Arduino IDE, placa Arduino Uno):

```
/*
 * ScoreSmash — Smart Tennis Arena (PI 6º CCOMP · FECAP · 2026)
 * Firmware da moldura de detecção de impacto
 * Entrega 2 — Sistemas Embarcados e Robótica
 *
 * Sensor  : 16 LDRs (8 no eixo X + 8 no eixo Y) lidos por 1 multiplexador 74HC4067 em
A0
 * Atuador : 81 LEDs (matriz 9 x 9) controlados por 10 registradores 74HC595
 *           em 2 grupos de 5 CIs em cascata (grupo A: fileiras 1 a 5 | grupo B:
fileiras 6 a 9 + 9ª coluna)
 * Placa   : Arduino Uno
 *
 * Funcionamento: cada LDR recebe o feixe de um laser. Quando a bola cruza a moldura,
ela corta
 * pelo menos um feixe vertical (X) e um horizontal (Y). O firmware descobre quais
feixes foram
 * cortados, calcula a posição na matriz 9 x 9, acende só o LED dessa posição e envia o
evento
 * pela Serial.
 *
 * Comandos pela Serial (115200 baud):
 *   S = parada segura (para a detecção e apaga os LEDs)
 *   R = reinicia (recalibra os sensores e volta a detectar)
 */

// ============================ PINOS ============================
// 74HC595 — grupo A (CIs 1 a 5)
const uint8_t A_SH = 7;    // SH_CP (pino 11) - clock
const uint8_t A_ST = 8;    // ST_CP (pino 12) - latch
const uint8_t A_DS = 9;    // DS    (pino 14) - dados
// 74HC595 — grupo B (CIs 6 a 10)
const uint8_t B_SH = 10;   // SH_CP (pino 11) - clock
const uint8_t B_ST = 11;   // ST_CP (pino 12) - latch
const uint8_t B_DS = 12;   // DS    (pino 14) - dados
// 74HC4067: S0..S3 em D2..D5 (bits 2 a 5 do PORTD) | SIG em A0
const uint8_t PIN_MUX_SIG = A0;

// ========================== PARÂMETROS ==========================
const uint8_t  N_FEIXES       = 8;     // feixes por eixo
const uint8_t  N_SENSORES     = 16;    // C0–C7 = eixo X (LDRs de baixo) | C8–C15 = eixo
Y (LDRs da direita)
const uint8_t  N_LEDS_LADO    = 9;     // matriz de LEDs 9 x 9
const uint8_t  CHIPS_GRUPO    = 5;     // 74HC595 por grupo
const float    FATOR_LIMIAR   = 0.60;  // feixe cortado se a leitura cair abaixo de 60%
da referência
const uint16_t LEITURA_MINIMA = 300;   // referência abaixo disso = laser fora do LDR
(desalinhado)
const uint16_t JANELA_US      = 6000;  // janela (µs) para juntar os cortes de X e Y do
mesmo impacto
const uint16_t TEMPO_EXIBICAO = 1000;  // ms que o LED do impacto fica aceso
const uint16_t TEMPO_FALHA    = 2000;  // feixe cortado por mais tempo que isso =
obstrução ou defeito
const uint8_t  AMOSTRAS_CALIB = 64;    // leituras por sensor na calibração

// ========================== MÁQUINA DE ESTADOS ==========================
enum Estado { CALIBRANDO, PRONTO, CAPTURANDO, EXIBINDO, FALHA, PARADA };
Estado estado = CALIBRANDO;

uint16_t referencia[N_SENSORES];       // leitura de cada sensor com o laser incidindo
uint16_t limiar[N_SENSORES];           // abaixo deste valor o feixe está cortado
uint8_t  ledsA[CHIPS_GRUPO];           // 1 byte por CI do grupo A (ledsA[0] = CI ligado
```

```
no Arduino)
uint8_t  ledsB[CHIPS_GRUPO];           // 1 byte por CI do grupo B

uint8_t  mascaraX = 0, mascaraY = 0;   // bit i = 1 -> feixe i cortado durante a captura
uint32_t inicioCaptura = 0;            // micros() do primeiro corte
uint32_t inicioExibicao = 0;           // millis() em que o LED foi aceso
uint32_t inicioBloqueio = 0;           // millis() em que o feixe ficou cortado após a
exibição
bool     bloqueado = false;
uint32_t totalImpactos = 0;

// ====================================================================
//  SENSORES (LDR + 74HC4067)
// ====================================================================

// Seleciona o canal do multiplexador escrevendo S0..S3 (D2..D5) numa única instrução
inline void selecionarCanal(uint8_t canal) {
  PORTD = (PORTD & 0xC3) | (canal << 2);
}

// Lê o sensor ligado ao canal informado (0 a 1023)
uint16_t lerSensor(uint8_t canal) {
  selecionarCanal(canal);
  delayMicroseconds(4);                // tempo para o multiplexador e o divisor
estabilizarem
  return analogRead(PIN_MUX_SIG);
}

// Varre os 16 sensores e marca, bit a bit, quais feixes estão cortados em cada eixo
void varrer(uint8_t &cortadosX, uint8_t &cortadosY) {
  cortadosX = 0;
  cortadosY = 0;
  for (uint8_t i = 0; i < N_FEIXES; i++) {
    if (lerSensor(i) < limiar[i])                       cortadosX |= (1 << i);
    if (lerSensor(i + N_FEIXES) < limiar[i + N_FEIXES]) cortadosY |= (1 << i);
  }
}

// Mede a leitura de cada sensor com o laser incidindo e define o limiar de corte de
cada um
bool calibrar() {
  bool ok = true;
  for (uint8_t canal = 0; canal < N_SENSORES; canal++) {
    uint32_t soma = 0;
    for (uint8_t a = 0; a < AMOSTRAS_CALIB; a++) soma += lerSensor(canal);
    referencia[canal] = soma / AMOSTRAS_CALIB;
    limiar[canal] = referencia[canal] * FATOR_LIMIAR;

    if (referencia[canal] < LEITURA_MINIMA) {        // LDR sem laser: desalinhado ou
laser queimado
      ok = false;
      Serial.print(F("{\"evt\":\"sensor_sem_laser\",\"canal\":"));
      Serial.print(canal);
      Serial.print(F(",\"leitura\":"));
      Serial.print(referencia[canal]);
      Serial.println('}');
    }
  }
  return ok;
}

// Converte os feixes cortados de um eixo (0 a 7) em uma posição da matriz de LEDs (0 a
8).
// A bola (6,7 cm) corta 1 ou 2 feixes (5 cm entre eles): a média dos índices dá o
centro,
// e a escala 8/7 leva os 8 feixes para as 9 posições de LED. Retorna -1 se nada foi
cortado.
```

```
int8_t posicao(uint8_t mascara) {
  uint16_t soma = 0, qtd = 0;
  for (uint8_t i = 0; i < N_FEIXES; i++) {
    if (mascara & (1 << i)) { soma += i; qtd++; }
  }
  if (qtd == 0) return -1;
  // round(media * 8 / 7) usando só inteiros
  return (2 * soma * (N_LEDS_LADO - 1) + (N_FEIXES - 1) * qtd) / (2 * (N_FEIXES - 1) *
qtd);
}

// ====================================================================
//  ATUADORES (81 LEDs + 10 x 74HC595 em 2 grupos)
// ====================================================================
//  Fileiras 1 a 5 (y = 0..4) -> grupo A, CI 1..5, saídas Q0..Q7 = colunas 1 a 8
//  Fileiras 6 a 9 (y = 5..8) -> grupo B, CI 1..4, saídas Q0..Q7 = colunas 1 a 8
//  Coluna 9 (x = 8)          -> grupo B, CI 5, saída Q(y) para as fileiras 1 a 8
//  O LED da fileira 9, coluna 9 não tem saída livre (10 CIs = 80 saídas para 81 LEDs).

// Envia os dados aos dois grupos juntos: o mesmo pulso de clock e de latch vale para os
dois,
// e cada grupo recebe o seu próprio bit em DS.
void atualizarLeds() {
  digitalWrite(A_ST, LOW);
  digitalWrite(B_ST, LOW);
  for (int8_t i = CHIPS_GRUPO - 1; i >= 0; i--) {         // último byte enviado fica no
1º CI
    for (int8_t b = 7; b >= 0; b--) {
      digitalWrite(A_DS, (ledsA[i] >> b) & 1);
      digitalWrite(B_DS, (ledsB[i] >> b) & 1);
      delayMicroseconds(5);
      digitalWrite(A_SH, HIGH); digitalWrite(B_SH, HIGH);
      delayMicroseconds(5);
      digitalWrite(A_SH, LOW);  digitalWrite(B_SH, LOW);
    }
  }
  digitalWrite(A_ST, HIGH); digitalWrite(B_ST, HIGH);       // copia para as saídas
  delayMicroseconds(5);
  digitalWrite(A_ST, LOW);  digitalWrite(B_ST, LOW);
}

void apagarLeds() {
  memset(ledsA, 0, sizeof(ledsA));
  memset(ledsB, 0, sizeof(ledsB));
  atualizarLeds();
}

// Acende só o LED da coluna x e fileira y (0 a 8). Retorna false se o LED não tem
saída.
bool acenderSomente(uint8_t x, uint8_t y) {
  memset(ledsA, 0, sizeof(ledsA));
  memset(ledsB, 0, sizeof(ledsB));
  bool ok = true;
  if (x < 8 && y < 5)       ledsA[y]     |= (1 << x);
  else if (x < 8)           ledsB[y - 5] |= (1 << x);
  else if (y < 8)           ledsB[4]     |= (1 << y);
  else                      ok = false;
  atualizarLeds();
  return ok;
}

// ====================================================================
//  COMUNICAÇÃO (eventos em JSON pela Serial)
// ====================================================================

void enviarEvento(const char *evt) {
  Serial.print(F("{\"t\":"));
```

```
  Serial.print(millis());
  Serial.print(F(",\"evt\":\""));
  Serial.print(evt);
  Serial.println(F("\"}"));
}

void enviarImpacto(int8_t x, int8_t y) {
  Serial.print(F("{\"t\":"));
  Serial.print(millis());
  Serial.print(F(",\"evt\":\"impacto\",\"x\":"));
  Serial.print(x);
  Serial.print(F(",\"y\":"));
  Serial.print(y);
  Serial.print(F(",\"n\":"));
  Serial.print(totalImpactos);
  Serial.println('}');
}

// Lê comandos recebidos: S = parada segura, R = reiniciar
void lerComandos() {
  while (Serial.available()) {
    char c = toupper(Serial.read());
    if (c == 'S') {
      estado = PARADA;
      apagarLeds();
      enviarEvento("parada_segura");
    } else if (c == 'R') {
      estado = CALIBRANDO;
      enviarEvento("reiniciando");
    }
  }
}

// ============================================================================
//  SETUP E LOOP
// ============================================================================

void setup() {
  Serial.begin(115200);

  // 74HC595: 6 pinos de saída, começando em LOW
  const uint8_t saidas[] = {A_SH, A_ST, A_DS, B_SH, B_ST, B_DS};
  for (uint8_t i = 0; i < 6; i++) { pinMode(saidas[i], OUTPUT); digitalWrite(saidas[i],
LOW); }
  pinMode(13, INPUT);                   // porta 13 não é usada

  // 74HC4067: D2..D5 como saída
  DDRD |= 0x3C;

  // ADC mais rápido: prescaler 16 (~16 µs por leitura em vez de ~112 µs).
  // Uma varredura dos 16 sensores cai de ~1,8 ms para ~0,35 ms.
  ADCSRA = (ADCSRA & ~0x07) | 0x04;

  // Autoteste dos atuadores: percorre a matriz acendendo um LED por vez
  for (uint8_t y = 0; y < N_LEDS_LADO; y++)
    for (uint8_t x = 0; x < N_LEDS_LADO; x++) { acenderSomente(x, y); delay(30); }
  apagarLeds();
}

void loop() {
  lerComandos();
  uint8_t cx, cy;

  switch (estado) {

    // Mede a referência de cada sensor. Se algum LDR estiver sem laser, entra em falha.
    case CALIBRANDO:
```

```
  apagarLeds();
  totalImpactos = 0;
  bloqueado = false;
  if (calibrar()) {
    enviarEvento("pronto");
    estado = PRONTO;
  } else {
    enviarEvento("falha_calibracao");
    estado = FALHA;
  }
  break;

// Varre continuamente. O primeiro feixe cortado inicia a captura do impacto.
case PRONTO:
  varrer(cx, cy);
  if (cx || cy) {
    mascaraX = cx;
    mascaraY = cy;
    inicioCaptura = micros();
    estado = CAPTURANDO;
  }
  break;

// Durante a janela, acumula todos os feixes cortados (a bola pode cortar X e Y em
// varreduras diferentes). No fim da janela calcula a posição.
case CAPTURANDO:
  varrer(cx, cy);
  mascaraX |= cx;
  mascaraY |= cy;
  if (micros() - inicioCaptura >= JANELA_US) {
    int8_t x = posicao(mascaraX);
    int8_t y = posicao(mascaraY);
    if (x >= 0 && y >= 0) {           // cortou os dois eixos: impacto válido
      totalImpactos++;
      acenderSomente(x, y);
      enviarImpacto(x, y);
    } else {                           // só um eixo: bola passou na borda
      enviarEvento("impacto_parcial");
    }
    inicioExibicao = millis();
    estado = EXIBINDO;
  }
  break;

// Mantém o LED aceso e só volta a detectar quando todos os feixes estiverem livres.
// Se algum feixe continuar cortado por muito tempo, há obstrução ou defeito.
case EXIBINDO:
  if (millis() - inicioExibicao >= TEMPO_EXIBICAO) {
    varrer(cx, cy);
    if (cx || cy) {
      if (!bloqueado) { bloqueado = true; inicioBloqueio = millis(); }
      else if (millis() - inicioBloqueio > TEMPO_FALHA) {
        enviarEvento("falha_feixe_bloqueado");
        estado = FALHA;
      }
    } else {
      bloqueado = false;
      apagarLeds();
      estado = PRONTO;
    }
  }
  break;

// Falha: o LED central pisca a 2 Hz até o comando R (um único LED, sem sobrecarga).
case FALHA:
  if ((millis() / 250) % 2) acenderSomente(4, 4);
  else                      apagarLeds();
```

```
      break;

    // Parada segura: nada é lido nem acionado até o comando R.
    case PARADA:
      break;
  }
}
```